Module 2: DATA EXTRACTION

Course Code: 22DS102006 - Feature Selection, Decision Trees, SVD & PCA

1. Extracting Meaning from Data

Organizations adopt two prominent philosophical frameworks for extracting meaning from big data: the crowdsourced competitive engineering paradigm pioneered by William Cukierski (Kaggle) and the mixed-methods behavioral paradigm advocated by David Huffaker (Google).

A) William Cukierski's Kaggle Paradigm

- **Feature Extraction:** Transforming raw, unstructured observations into well-formed mathematical feature spaces to avoid garbage-in, garbage-out modeling.
- **Feature Selection:** Isolating the most informative subsets of variables to minimize model complexity, eliminate collinearity, and boost generalization.
- **Crowdsourced Competitions:** Kaggle distributes anonymized training datasets with withheld test targets, fostering rapid iterations on a dynamic public/private leaderboard.
- **The Leapfrogging Effect:** Competitors continuously outdo prior solutions, driving breakthrough architectures (e.g. Netflix Prize). However, prolonged competitions can yield overly complex ensembles difficult to deploy in production.

The Dynamics of Leapfrogging & Industry Collaborations

- **Economic Leapfrogging:** Developing economies bypassing legacy infrastructures (e.g. skipping wire landlines directly to 5G cellular finance).
- **Technological Leapfrogging:** Skipping iterative software generations to deploy state-of-the-art transformer pipelines directly.
- **Corporate Impact:** Allstate improved its actuarial risk prediction accuracy by 271% through Kaggle crowdsourcing; small prize pools unlocked immense corporate IP.
- **Recruitment & Ethical Considerations:** Competitions serve as talent discovery platforms, but raise questions regarding unpaid contributor value and corporate exploitation.

B) David Huffaker's Google Paradigm (Mixed Methods)

- **Mixed-Methods Synergy:** Combining large-scale quantitative clickstream telemetry with in-depth qualitative user interviews and ethnographic observation.
- **From Description to Prediction:** Evolving descriptive dashboards into predictive models of user satisfaction and long-term retention.
- **Ethics & Informed Consent:** Prioritizing transparency, user privacy controls, and explainable models over black-box scoring.

2. Feature Selection Strategies

Features are measurable attributes that describe observed phenomena. Feature selection systematically removes redundant, noisy, or irrelevant features.

Why Feature Selection is Imperative:

- **Mitigating Overfitting:** Simpler models with fewer degrees of freedom generalize significantly better to unseen test distributions.
- **Computational Efficiency:** Fewer dimensions translate to vastly reduced training times and lower RAM footprints during batch/real-time inference.
- **Model Interpretability:** Enables stakeholders to understand the true causal drivers behind predictions.

The Four Feature Selection Approaches:

- **1. Filter Methods:** Ranks features based on statistical association independent of model training (Pearson correlation, Chi-Square test, Mutual Information). Fast and computationally lightweight.
- **2. Wrapper Methods:** Evaluates feature subsets by repeatedly training and testing a chosen estimator (e.g. Recursive Feature Elimination - RFE). Computationally intensive but high performance.
- **3. Embedded Methods:** Feature selection occurs natively during model optimization (e.g. LASSO L1 regularization driving coefficients to zero, decision tree split rankings).
- **4. Hybrid Methods:** Filters out 90% of noise variables via filter methods first, then optimizes remaining subsets using wrapper routines.

Stepwise Regression & Model Selection Criteria

Stepwise regression systematically tests combinations of features through sequential additions and deletions.

- **Forward Selection:** Begins with an intercept-only model; iteratively adds the feature that yields the greatest reduction in residual sum of squares.
- **Backward Elimination:** Begins with all candidate variables; iteratively removes the least statistically significant predictor until all remaining p-values meet alpha threshold.
- **Bidirectional (Stepwise):** Alternates forward entry and backward pruning to ensure no redundant predictors are retained.

Model Evaluation Criteria:

- **R-squared (R^2):** Proportion of target variance explained by the model.
- **Adjusted R^2:** Penalizes the addition of non-informative variables that artificially inflate raw R^2.
- **Akaike Information Criterion (AIC):** Balances model likelihood against parameter count (AIC = 2k - 2ln(L)). Lower AIC indicates superior efficiency.
- **Bayesian Information Criterion (BIC):** Imposes a steeper penalty on parameter count based on sample size ln(n).
- **Cross-Validation Error:** K-fold cross-validation estimates out-of-sample generalization error directly.

3. User Retention Modeling & Decision Trees

In product data science, predictive retention models forecast which user actions lead to long-term loyalty versus churn.

Interpretability vs. Predictive Power

High-capacity neural networks can predict churn with high AUC, but fail to explain why. Decision trees offer transparent rule paths (e.g., 'If session_length < 2 min and ad_views >= 3 -> Churn probability = 87%'). A/B testing is crucial to distinguish genuine causality from correlation.

4. Entropy & Information Theory

Entropy measures the level of disorder, impurity, or uncertainty in a probability distribution.

Formula for a Binary Variable X in {0, 1}:

H(X) = - p(X=1) * log2 p(X=1) - p(X=0) * log2 p(X=0)

Key Properties:
- Zero Entropy (H = 0): Completely pure node (e.g. 100% yes, 0% no). Zero uncertainty.
- Maximum Entropy (H = 1.0 bit): Completely impure node (50% yes, 50% no). Peak uncertainty.

5. Decision Tree Algorithm

Decision trees recursively partition feature space into homogeneous rectangular regions through a sequence of binary conditions.

Core Components:

- **Root Node:** The top-most node containing the entire dataset, split on the feature yielding maximum Information Gain.
- **Internal Nodes:** Decision decision points checking feature thresholds (e.g. income > 50,000).
- **Branches:** Outcomes of decision checks leading to child nodes.
- **Leaf Nodes:** Terminal nodes providing final class predictions or regression value averages.

Handling Continuous Variables:

- **Threshold Sorting:** Sort unique continuous values in ascending order and test midpoints as candidate split boundaries.
- **Information Gain Maximization:** Select the split point c that maximizes: Information Gain = H(Parent) - [(N_left/N)*H(Left) + (N_right/N)*H(Right)].
- **Tree Pruning:** Pre-pruning limits max_depth and min_samples_split; post-pruning applies cost-complexity pruning (alpha) to prevent overfitting.

6. Random Forests & Ensemble Bagging

Random Forest is an ensemble learning method that constructs a multitude of uncorrelated decision trees at training time and aggregates their individual predictions via majority voting (classification) or averaging (regression).

Key Architectural Mechanisms:

- **Bootstrap Aggregating (Bagging):** Each tree trains on a random bootstrap sample drawn with replacement (~63.2% unique records, ~36.8% out-of-bag).
- **Random Feature Subspacing:** At each split node, only a random subset of features F (typically sqrt(p) for classification, p/3 for regression) is considered. This decorrelates individual trees.
- **Full Depth Growth:** Individual trees are allowed to grow deep without pruning; their high variance is neutralized when hundreds of trees are averaged.

Implementation Example in R (Diamonds Dataset)

```
# Load libraries and diamond price data
require(ggplot2)
require(glmnet)
require(rpart)
require(randomForest)

data(diamonds)
diamonds$Expensive <- ifelse(diamonds$price >= 12000, 1, 0)
diamonds$price <- NULL

# Train Decision Tree via rpart
modTree <- rpart(Expensive ~ ., data = diamonds)

# Train Random Forest Classifier
modForest <- randomForest(as.factor(Expensive) ~ ., data = diamonds,
                          ntree = 200, mtry = 3, importance = TRUE)
print(importance(modForest))
```

Pros and Cons of Random Forests:

- **Pros:** Exceptional predictive accuracy, highly resistant to overfitting, built-in feature importance estimation, handles missing data.
- **Cons:** Black-box nature compared to single tree, high computational and memory cost for massive datasets.

7. Singular Value Decomposition (SVD)

Singular Value Decomposition (SVD) is a fundamental theorem in linear algebra stating that any real m x n matrix X of rank k can be factorized into the product of three constituent matrices:

X = U * S * V^T

Where:
- U is an m x k orthogonal matrix whose columns are the left singular vectors (eigenvectors of X * X^T).
- S is a k x k diagonal matrix with non-negative real singular values sorted in descending order: sigma_1 >= sigma_2 >= ... >= sigma_k > 0.
- V is an n x k orthogonal matrix whose columns are the right singular vectors (eigenvectors of X^T * X).

Applications in Data Science:

- **Low-Rank Approximation:** Truncating S to the top d singular values (d < k) creates the optimal rank-d approximation of X under the Frobenius norm (Eckart-Young Theorem).
- **Recommender Systems:** Decomposes user-item rating matrices into latent user preferences (U) and latent item characteristics (V).

8. Principal Component Analysis (PCA)

Principal Component Analysis is the premier unsupervised linear dimensionality reduction technique. It projects high-dimensional data onto orthogonal axes (principal components) that capture the maximum possible variance while eliminating covariance.

Step-by-Step Mathematical Algorithm:

- **Step 1: Mean Centering:** Standardize each feature column to have mean zero and unit variance (Z-score normalization).
- **Step 2: Covariance Matrix:** Compute covariance matrix Sigma = (1/n) * X^T * X.
- **Step 3: Eigen-Decomposition:** Calculate eigenvalues lambda_i and eigenvectors v_i such that Sigma * v_i = lambda_i * v_i.
- **Step 4: Sorting & Selection:** Order eigenvectors by eigenvalues descending; choose the top d eigenvectors explaining >= 90-95% of cumulative variance.
- **Step 5: Projection:** Transform original data: Z = X * V_d.

Core Advantages & Benefits of PCA:

- **Curse of Dimensionality:** Compresses dozens/hundreds of features into a compact uncorrelated feature set.
- **Visual Exploration:** Projects multi-dimensional clusters into 2D/3D scatter plots.
- **Noise Attenuation:** Discards components with near-zero eigenvalues which typically represent measurement noise.